Data Structures

Part 3: Dictionaries

Using key-value pairs instead of indices to store grouped data.

Dictionaries and Key Value Pairs

A dictionary is a data structure that stores information as key-value pairs.

Each key acts as a unique name used to access its associated value instead of using a numerical index.

Values can be different data types and can be added, removed, or changed after the dictionary is created.

Dictionaries are useful when each piece of data has a meaningful label.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}
```

Creating a Dictionary

A dictionary is created using curly brackets `{}` to surround its contents.

Inside the `{}`, each item is written as a key-value pair, with a colon (`:`) separating the key from its value.

Multiple key-value pairs are separated by commas (`,`).

An empty dictionary can be created using just `{}`.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}
```

Accessing Values with Keys

Values in a dictionary are accessed using their key instead of a numerical index.

Place the key inside square brackets `[]` after the dictionary's variable name.

Python then returns the value associated with that key.

Each key in a dictionary must be unique.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

print(student["name"])
print(student["grade"])
```

OUTPUT

```
Alice
11
```

Changing Values with Keys

Dictionary values can be changed by accessing an existing key and assigning it a new value.

The key is placed inside square brackets `[]`, followed by the assignment operator `=` and the new value.

This changes the value associated with that key while leaving the rest of the dictionary unchanged.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}
student["average"] = 92.0
print(student)
```

```
OUTPUT
{'name': 'Alice', 'grade': 11, 'average': 92.0}
```

Adding New Key-Value Pairs

A new key-value pair can be added to a dictionary by placing a new key inside square brackets `[]` and assigning it a value.

If the key does not already exist, Python creates a new key-value pair in the dictionary.

If the key already exists, its current value will be replaced instead.

This makes dictionaries easy to expand as new information becomes available.

```
student = {
    "name": "Alice",
    "grade": 11
}

student["average"] = 87.5

print(student)
```

```
OUTPUT
{'name': 'Alice', 'grade': 11, 'average': 87.5}
```

Removing Items – pop()

The **pop()** method removes a key-value pair from a dictionary using its key.

It also returns the value that was removed, allowing it to be stored or used elsewhere.

This is useful when you know which key you want to remove.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

removed = student.pop("average")

print(removed)
print(student)
```

OUTPUT

```
87.5
{'name': 'Alice', 'grade': 11}
```

Removing Items – popitem()

The **popitem()** method removes the last key-value pair added to a dictionary.

It returns the removed key and value together as a tuple.

This is useful when you want to remove the most recently added item without specifying its key.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

removed = student.popitem()

print(removed)
print(student)
```

OUTPUT

```
('average', 87.5)
{'name': 'Alice', 'grade': 11}
```

Removing Items – `clear()`

The **`clear()`** method removes all key-value pairs from a dictionary.

The dictionary still exists afterward, but it is empty.

This is useful when you want to reset a dictionary and continue using the same variable.

```
student = {
    "name": "Alice",
    "grade": 11
}

student.clear()

print(student)
```

```
OUTPUT
{}
```

Checking Length with `len()`

The `len()` function returns the number of key-value pairs stored in a dictionary.

Each pair counts as one item.

This is useful when you need to know how many pieces of information are stored.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

print(len(student))
```

```
OUTPUT
3
```

Dictionary Specific Methods – `get()`

The `get()` method retrieves the value associated with a specified key.

Unlike square brackets `[]`, it does not cause an error if the key does not exist.

You can also provide a default value to return when the key is missing.

```
student = {
    "name": "Alice",
    "grade": 11
}

print(student.get("name"))

# Setting a default value
print(student.get("average", "N/A"))
```

OUTPUT

```
Alice
N/A
```

Dictionary Specific Methods – `setdefault()`

The **`setdefault()`** method checks whether a key already exists in a dictionary.

If the key exists, its current value is returned; if it does not exist, the key is added with a specified default value.

This is useful when you want to ensure that a dictionary contains a particular key without replacing an existing value.

```
student = {
    "name": "Alice",
    "grade": 11
}

student.setdefault("average", 0)

print(student)
```

OUTPUT

```
{'name': 'Alice', 'grade': 11, 'average': 0}
```

Dictionary Specific Methods – keys()

The **keys()** method provides access to all of the keys stored in a dictionary.

It is commonly used when you want to traverse the labels or names used by the dictionary.

The keys can be accessed with a **for** loop.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

for key in student.keys():
    print(key)
```

OUTPUT

```
name
grade
average
```

Dictionary Specific Methods – `values()`

The **`values()`** method provides access to all of the values stored in a dictionary.

It is useful when you only need the stored data and do not need the keys.

The values can be traversed with a **`for`** loop.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

for value in student.values():
    print(value)
```

OUTPUT

```
Alice
11
87.5
```

Dictionary Specific Methods – `items()`

The **`items()`** method provides access to both the keys and values stored in a dictionary.

Each key-value pair can be unpacked into two variables while traversing the dictionary.

This is useful when you need to work with both parts of each dictionary entry.

```
student = {
    "name": "Alice",
    "grade": 11,
    "average": 87.5
}

for key, value in student.items():
    print(f"key: {key}, value: {value}")
```

OUTPUT

```
key: name, value: Alice
key: grade, value: 11
key: average, value: 87.5
```

Copy vs Reference

When you assign an existing dictionary to another variable, Python does not create a new dictionary.

Instead, the new variable becomes a reference to the same dictionary, meaning both variables point to the same data in memory and changes made through either variable affect both.

Using the `copy()` method creates a separate dictionary that can be changed without changing the original.

This is useful when you want to work with a modified version of a dictionary while keeping the original unchanged.

```
original = {"name": "Alice", "grade": 11}

reference = original
copied = original.copy()

reference["grade"] = 12
copied["name"] = "Bob"

print(f"Original: {original}")
print(f"Reference: {reference}")
print(f"Copy: {copied}")
```

OUTPUT

```
Original: {'name': 'Alice', 'grade': 12}
Reference: {'name': 'Alice', 'grade': 12}
Copy: {'name': 'Bob', 'grade': 11}
```